

RynnValue: Scaling Robotic Value Foundation Models with Temporal Distance

Dongchi Huang*, Hongyin Zhang*, Bohan Hou*, Siteng Huang[†], Zhian Su, Hang Guo, Tong Lu, Zhaofeng Xu, Jiahao Tang, Jianfei Yang, Donglin Wang, Peixi Peng[†], Mingxiu Chen, Deli Zhao, Xin Li

¹DAMO Academy, Alibaba Group ²Hupan Lab

*Equal contribution, [†]Corresponding authors

General-purpose reward models are increasingly the bottleneck for scaling robot learning, yet the recipe for learning value-related capabilities from large-scale heterogeneous corpora remains underexplored. Existing approaches tie supervision to task-internal anchors such as preferences or normalized progress, none of which transfer cleanly across embodiments and data sources. We introduce **RynnValue**, an open-source value foundation model for robotic manipulation that replaces these anchors with temporal distance, the directed cost-to-go from an observation to the language-specified goal. Because temporal-distance labels can be derived directly from timestamps, RynnValue scales to over 7,000 hours and roughly 3M instruction-conditioned clips without preference or progress annotations. To make temporal-value learning reliable at scale, we combine random temporal sampling, temporal-order shuffling, and value-isolation attention, suppressing shortcuts that would leave predictions insensitive to failures and regressions. Trained without preference labels, RynnValue attains an average Kendall's τ_a of 0.675 on RBM-EVAL-ODD, surpassing the fully preference-supervised state of the art (0.655) and more than doubling a progress-only counterpart (0.292), while generalizing zero-shot to unseen tasks, embodiments, and viewpoints. Converted into dense rewards via potential-based shaping, it raises real-world policy success from 52.5% to 72.5% online and from 63.8% to 82.5% offline. These results establish temporal distance as a scalable supervision target and practical reward interface for generalist robot policies.



<https://alibaba-damo-academy.github.io/RynnValue.github.io>



<https://github.com/alibaba-damo-academy/RynnValue>



<https://huggingface.co/collections/Alibaba-DAMO-Academy/rynnvalue>



https://www.modelscope.cn/collections/DAMO_Academy/RynnValue

Date: August 10, 2026



1 Introduction

Generalist robot policies are increasingly trained with reinforcement learning [2, 12], yet scaling this loop is bottlenecked not by policy capacity but by reward supervision. Hand-designed rewards hardly generalize to open-ended tasks, while sparse success signals offer limited guidance for long-horizon behavior [31]. Existing general-purpose reward models rely on task-internal anchors such as preferences, reference demonstrations, or local state comparisons [31, 34, 18, 26], which tie supervision to particular trajectories or comparison sets and are difficult to reuse across heterogeneous data. The common fallback, normalized [0,1] progress [16, 15], is an intra-trajectory coordinate rather than a goal-conditioned cost-to-go [25, 24], making it poorly aligned with the standard notion of value in control and difficult to maintain consistently across varying durations, embodiments, and task structures.

We argue for a shift in framing: from a reward model scoring trajectory-level anchors to a **value foundation model** predicting goal-conditioned cost-to-go as a single reusable interface. We instantiate this view in **RynnValue**, which adopts *temporal distance* as its scaling target. Rather than normalizing progress within a trajectory, RynnValue estimates the directed temporal cost from the current observation to the language-specified goal. Under a minimum-time objective, this corresponds to the hitting-time cost-to-go, yielding clear



Figure 1 Overview of RynnValue. RynnValue is a language-conditioned value model trained on over 7,000 hours of heterogeneous embodied data, comprising roughly 3M instruction-conditioned trajectory clips across diverse embodiments, viewpoints, and task domains. Given a language instruction and a sequence of sampled observations, the model builds an interleaved multimodal sequence of repeated absolute-value and relative-value queries, which RynnBrain encodes in a single forward pass. Two distributional heads then predict the absolute temporal distance to task completion and the signed relative temporal displacement between observations, while the language branch produces video analysis and language-conditioned verification. The resulting temporal values serve as a unified interface for progress estimation, failure detection, and reward specification in robotic reinforcement learning.

directionality and task conditioning. It can further be converted into dense rewards through potential-based shaping [22], serving as a practical value interface rather than a task-specific heuristic.

Crucially, temporal distance scales naturally to heterogeneous data, since labels can be derived directly from timestamps once a completion cutoff is identified. Using this recipe, we construct a large training mixture of real-world, simulated, and egocentric trajectories spanning diverse embodiments, viewpoints, and task families. After preprocessing, the corpus exceeds 7,000 hours and comprises approximately 3M instruction-conditioned clips. Raw episode boundaries often do not reflect semantic task completion; we therefore apply subtask segmentation and cutoff relabeling to define coherent completion points. Unlike progress and comparison-based supervision [16], this recipe requires only instructions, timestamps, and relabeled cutoffs, without constructing preference pairs or progress annotations.

Scaling data alone, however, does not guarantee reliable temporal-value learning. In a multi-frame setting, the model may exploit regular sampling intervals, the presentation order of observations, or other value-query representations, rather than grounding predictions in task-relevant visual evidence. Such shortcuts can leave learned values insensitive to regressions, failures, and other non-monotonic events [8]. RynnValue therefore incorporates complementary designs at two levels. At the visual level, we sample observations at irregular timestamps and perturb their temporal order during training, breaking the correspondence between sequence position and temporal distance. At the value level, we introduce value-isolation attention, so that each value-query group attends only to its own language-visual context and cannot access the value queries of other observations. Together, these designs suppress both temporal-order and value-extrapolation shortcuts, forcing each prediction to rely on the corresponding visual observation and task semantics. RynnValue further jointly supervises natural-language video description, instruction matching, and success prediction, adding sequence-level semantic cues alongside continuous temporal-value learning.

We evaluate RynnValue both as a standalone value model and as a reward interface for downstream policy learning. Trained without preference labels, RynnValue-8B attains an average Kendall’s τ_a of 0.675 on RBM-EVAL-OOD, surpassing the fully preference-supervised state of the art (0.655) and more than doubling a progress-only counterpart (0.292), while generalizing zero-shot to unseen tasks, embodiments, and viewpoints. Converted into dense rewards via potential-based shaping, it raises real-world policy success from 52.5% to 72.5% online and from 63.8% to 82.5% offline over the strongest baseline.

In summary, our work makes the following contributions:

- We identify normalized progress as a supervision bottleneck for general-purpose robotic reward modeling, and reframe the problem as learning a **value foundation model** whose scaling target is **temporal distance**

(cost-to-go), a scalable, preference-free value target that unifies heterogeneous data under a single interface.

- We present **RynnValue**, a robotic value foundation model trained with a label-cheap data recipe: labels are derived directly from timestamps, aided by subtask segmentation and cutoff relabeling, over 7,000 hours and roughly 3M instruction-conditioned clips spanning diverse embodiments, viewpoints, and task families.
- We introduce complementary shortcut-suppression designs, **temporal-order shuffling** and **value-isolation attention**, that keep temporal-value predictions grounded in visual evidence rather than spurious correlations, and pair them with dual distributional heads for absolute and relative temporal values.
- We show that RynnValue **surpasses preference-supervised state-of-the-art** on out-of-distribution trajectory ranking without any preference annotation, and via potential-based shaping, serves as a dense reward interface that improves both online and offline real-world policy learning.

2 Model Architecture

As shown in Figure 1, RynnValue is a language-conditioned visual value model that estimates the temporal distance from robot observations to task completion. RynnValue is built on RynnBrain [5] and further pretrained on large-scale robot data, which provides representations adapted to embodied scenes, robot states, and language-conditioned manipulation tasks.

Given an instruction ℓ , which includes the task instruction and system prompt of absolute/relative temporal questions, embodiment metadata m , and a sequence of K observations $\mathcal{I} = \{I_{t_i}\}_{i=1}^K$, RynnValue jointly predicts the absolute temporal distance of each observation and the relative temporal displacement between consecutive observations. In our main setting, we sample $K = 8$ observations from each video. We then describe the notable elements in the architecture design.

Grouped temporal queries. A single query token is an information bottleneck for temporal-distance estimation, which must summarize several visual cues at once: object configuration, robot-object interaction, intermediate task stages, and task-completion evidence. Following [11], we therefore represent each temporal prediction using a group of N repeated query tokens, allowing the model to construct a richer and more robust temporal representation.

For each observation I_{t_i} , we insert an absolute-value query group $\mathbf{V}_i = [V_{i,1}, \dots, V_{i,N}]$, whose tokens share the `<value>` query type. Starting from the second observation, we additionally insert a relative-value query group $\mathbf{R}_{i-1} = [R_{i-1,1}, \dots, R_{i-1,N}]$ before $\mathbf{V}_i$, whose tokens share the `<relative_value>` query type. In our implementation, we use $N = 8$ repeated tokens for both absolute- and relative-value query groups.

After the final temporal-query group, we append a natural-language verification prompt $\mathbf{p}_{\text{ver}}$. The resulting multimodal sequence is

$$\mathbf{x} = [m, \ell, I_{t_1}, \mathbf{V}_1, I_{t_2}, \mathbf{R}_1, \mathbf{V}_2, \dots, I_{t_K}, \mathbf{R}_{K-1}, \mathbf{V}_K, \mathbf{p}_{\text{ver}}]. \quad (1)$$

The verification prompt is placed after all visual observations and temporal queries. It instructs the model to first generate a natural-language analysis of the video and subsequently determine task matching and task completion.

Tokens within the same temporal-query group interact through bidirectional attention and capture complementary aspects of the corresponding observation. Rather than averaging these representations, we concatenate the N query hidden states along the feature dimension:

$$\tilde{\mathbf{h}}_i^V = H_\theta(\mathbf{x})_{\text{pos}(V_{i,1})} \parallel \dots \parallel H_\theta(\mathbf{x})_{\text{pos}(V_{i,N})}, \quad \tilde{\mathbf{h}}_i^R = H_\theta(\mathbf{x})_{\text{pos}(R_{i,1})} \parallel \dots \parallel H_\theta(\mathbf{x})_{\text{pos}(R_{i,N})}, \quad (2)$$

where H_θ denotes the contextual representations produced by RynnBrain and $\parallel$ denotes feature concatenation. This operation preserves the complementary information captured by different query positions and produces an Nd -dimensional representation for each absolute or relative temporal prediction.

Continuous temporal readouts. RynnValue employs two specialized distributional heads to produce continuous absolute and relative temporal estimates:

$$\mathbf{z}_i^V = \text{ValueHead}(\tilde{\mathbf{h}}_i^V), \quad \mathbf{z}_i^R = \text{RelativeHead}(\tilde{\mathbf{h}}_i^R). \quad (3)$$

Here, $\mathbf{z}_i^V$ and $\mathbf{z}_i^R$ denote the bin logits produced by the absolute and relative distributional heads, respectively. Numerical prediction is performed exclusively by the two heads, while the original LM head is retained only for the subsequent natural-language video analysis and task verification.

Absolute and relative temporal estimates. The absolute target is the observed remaining time from an observation to its relabeled completion cutoff, while the relative target is the signed temporal displacement between adjacent observations in the presented sequence. Relative prediction provides a local temporal learning signal that complements the globally anchored absolute temporal distance. We discretize the absolute range $([0, 512])$ seconds and the relative range $([-256, 256])$ seconds into 256 symlog-spaced bins, and train distributional heads using two-hot targets over adjacent bins [9]. This casts temporal-distance regression as a stable classification problem: symlog binning spans the wide dynamic range of temporal distances by compressing large-magnitude tails while leaving near-zero targets essentially undistorted, and the two-hot encoding represents each continuous target exactly through its two nearest bins, decoupling gradient magnitude from target scale for stable, precise regression. At inference time, predictions are decoded by taking the expected bin center in symlog space and applying the inverse symlog transform [7]:

$$v_i = \text{symexp} \left(\sum_{b=1}^{|\mathcal{B}_V|} c_b^V \text{softmax}(\mathbf{z}_i^V)_b \right), \quad \Delta_i = \text{symexp} \left(\sum_{b=1}^{|\mathcal{B}_R|} c_b^R \text{softmax}(\mathbf{z}_i^R)_b \right), \quad (4)$$

where c_b^V and c_b^R are the absolute and relative bin centers in symlog space, respectively, and symexp denotes the inverse symlog transformation. The decoded v_i is the predicted absolute temporal distance, *i.e.*, the remaining time from observation i to task completion, whereas Δ_i is the predicted signed temporal displacement between the i -th and the next presented observation; both are continuous estimates even though their heads are trained with a distributional objective.

Value-isolation attention. Without an explicit attention constraint, a temporal query can predict its value by extrapolating from previously exposed value tokens rather than by interpreting the corresponding visual evidence. This shortcut produces smooth value curves while leaving predictions insensitive to regressions, failures, and other non-monotonic events. Inspired by attention masking strategies that explicitly control information flow and prevent target leakage [32, 6], we introduce value-isolation attention as shown in Figure 2(b), which prevents absolute and relative temporal queries associated with different observations from attending to one another. Queries belonging to the same observation remain mutually visible, allowing their features to interact before concatenation.

We further prevent context tokens from attending to temporal-query tokens, so that previous value predictions cannot be propagated indirectly through later language or visual representations. As a result, each temporal estimate must be grounded in the task instruction and its available visual context, while natural-language analysis and verification outputs are generated independently of the predicted values.

Natural-language video analysis and task verification. RynnValue additionally retains the autoregressive language capability of RynnBrain for interpretable task analysis. After processing the complete visual sequence, the verification prompt $\mathbf{p}_{\text{ver}}$ first instructs the model to generate a natural-language description of the video and its task-relevant events. The model then generates task-matching and task-success judgments:

$$\mathbf{y}_{\text{lang}} = [\text{Video Description: } \mathbf{y}^{\text{vid}}, \text{Match: } \mathbf{y}^{\text{match}}, \text{Success: } \mathbf{y}^{\text{succ}}]. \quad (5)$$

This autoregressive ordering encourages the model to identify the robot behavior, object interactions, task-relevant events, and completion evidence before producing the final matching and success judgments.

The natural-language outputs are generated from the task instruction and complete visual context through the original LM head. They are auxiliary outputs produced after the temporal-query sequence and are not fed back into the absolute or relative temporal heads, so the LM head does not participate in temporal-value prediction. RynnValue therefore supports interpretable video analysis, task matching, completion recognition, and failure detection without introducing separate classification heads.

Table 1 Composition of the heterogeneous data mixture before subtask expansion. The corpus contains 1.67M original episodes and is further converted into over 3M instruction-conditioned trajectory segments after subtask segmentation and cutoff relabeling.

Data Source	# Original Episodes	# Segmentations	# Instructions	Segmentation Source
AgiBot [3]	167,535	1,166,042	3,741	coarse task
EgoDex [10]	338,234	338,234	2,038	full trajectory
Galaxea Open-World [13]	16,979	95,671	11,070	coarse task
InternData-A1 [27]	320,905	320,905	348	full trajectory
Open X-Embodiment [23]	693,037	693,037	180,090	full trajectory
RDT [17]	6,109	6,109	272	per-file coarse task
RoboCOIN [30]	67,420	410,877	2,124	coarse task
RoboMIND [29]	32,138	32,138	184	full trajectory
RoboTwin [20]	27,414	27,414	23,527	full trajectory
Soft-FOLD [35]	1,542	1,542	1	per-file coarse task
Total	1,671,313	3,091,969	223,395	–

3 Learning Temporal Distance as a Reward Interface

This section describes how RynnValue learns temporal distance from heterogeneous data and exposes it as a reward interface. Our approach has three components: a heterogeneous data mixture and temporal-distance relabeling that turn raw trajectories into supervision (Section 3.1), a distributional training recipe (Section 3.2), and a reward interface for downstream policy learning (Section 3.3). We describe each in turn.

3.1 Data Preparation

RynnValue is trained from large-scale heterogeneous data. The goal is to convert trajectories with different task boundaries, embodiments, viewpoints, and execution speeds into a unified state-level temporal-distance supervision signal.

3.1.1 Heterogeneous Data Mixture

To scale temporal-distance learning beyond curated task-specific demonstrations, we train RynnValue on a large heterogeneous mixture of robot, simulation, and egocentric data [3, 10, 13, 23]. The mixture exceeds 7,000 hours of trajectories and spans a broad range of manipulation settings, including single-arm platforms, dual-arm mobile manipulators, bimanual tabletop robots, dexterous-hand systems, simulated embodiments, and first-person human demonstrations. It also covers diverse camera configurations. This diversity exposes the model to large variations in morphology, viewpoint, control frequency, execution speed, task duration, object distribution, and language annotation granularity.

Before subtask expansion, the corpus contains 1.67M original episodes. After subtask segmentation and cutoff-based relabeling, these data are further converted into over 3M instruction-conditioned trajectory segments, each associated with a language goal and a temporally localized completion target. Crucially, these sources are not unified by manually defining dataset-specific progress scales. Instead, they are mapped onto the same temporal-distance supervision interface: each trajectory segment provides timestamp-derived state-level labels measuring the remaining temporal distance to its completion cutoff.

3.1.2 Data Preprocessing and Temporal-Distance Relabeling

Large-scale robot datasets differ substantially in how episodes and subtasks are defined [3, 23, 4]. We therefore convert each data source into instruction-conditioned trajectory segments. Long demonstrations are split using native temporal annotations when available; otherwise, the complete episode is retained as a coarse segment. Because recorded trajectories may contain post-completion motions, we further assign each segment a completion cutoff. The segment endpoint is used by default, while dataset-specific ratio- or duration-based trimming is applied when necessary to better approximate the first semantically completed observation [16].

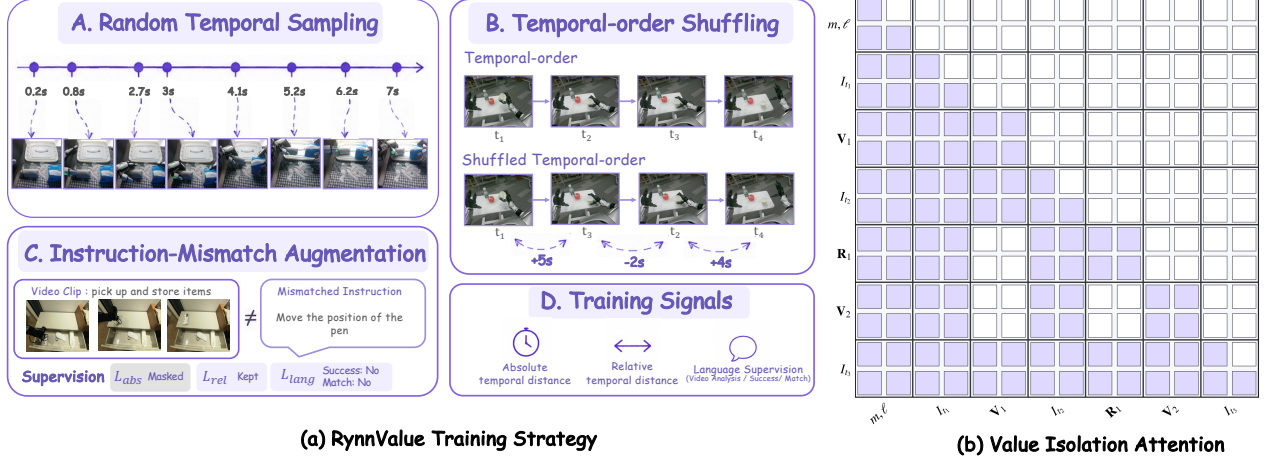


Figure 2 RyannValue training pipeline and value-isolation attention. **(a)** Training strategy. Random temporal sampling and temporal-order shuffling suppress shortcuts tied to sampling intervals and sequence position, while instruction-mismatch augmentation strengthens language–visual grounding. RyannValue jointly learns absolute temporal distance, relative temporal displacement, and natural-language supervision. **(b)** Value-isolation attention. Within each value-query group, repeated queries attend to one another and to the language–visual context, while remaining isolated from other value-query groups. Colored cells denote visible attention connections.

Temporal-distance labels are then generated directly from timestamps: observations before the cutoff are labeled by their remaining time to completion, while observations at or after the cutoff receive zero. This provides unified, dense supervision without dataset-specific progress normalization. Separately, we use Qwen3-VL-72B [1] to generate a segment-level description of the observed behavior and task-relevant events. These captions supervise the natural-language video-analysis output (*i.e.*, Video Description in Equation 5) and do not affect the temporal targets.

3.2 Training Recipe

3.2.1 Multi-Frame Sampling Strategy

Scaling data alone does not guarantee reliable temporal-value learning, because a multi-frame model can exploit regularities in the input rather than the visual evidence. We therefore combine two complementary sampling components: *random temporal sampling* removes regularities in the temporal intervals, while *temporal-order shuffling* breaks the correspondence between sequence position and task progress.

Random temporal sampling. For each instruction-conditioned trajectory clip, we randomly sample $K = 8$ observations at irregular timestamps. The resulting non-uniform temporal gaps break the near-arithmetic value patterns induced by uniform sampling and discourage the model from exploiting fixed sampling intervals as a shortcut.

Temporal-order shuffling. As illustrated in Figure 2(a), we shuffle the chronological order of sampled observations. Half of the training sequences are independently sampled without temporal sorting, while the remainder follow a forward-biased temporal walk with occasional backward transitions, whose per-step probability we refer to as the rewind probability. Relative temporal targets are computed between adjacent observations in the presented order and can therefore be either positive or negative.

Temporal-order shuffling and value-isolation attention suppress complementary shortcuts. Shuffling prevents the model from regressing a stereotypical value curve from sequence positions, while value-isolation attention prevents extrapolation from other value-query representations. Together, they require each prediction to be grounded in the corresponding visual observation and language-conditioned task semantics.

3.2.2 Instruction-Mismatch Augmentation

A value model should also recognize when the language instruction does not describe the observed video, rather than always reporting smooth progress toward the stated goal. As shown in Figure 2, we therefore introduce instruction-mismatch augmentation. For 10% of the training samples, we replace the original instruction with one sampled from a different trajectory and supervise the language branch to predict **Match: No** and **Success: No**. Because the original completion cutoff is no longer valid under the substituted instruction, we mask the absolute temporal-distance loss (Equation 6) to avoid introducing incorrect supervision. The relative temporal loss (Equation 7) is retained, as the relative target measures the temporal displacement between observations and is independent of the task instruction. This augmentation teaches RynnValue to detect instruction-video mismatches and remain reliable under substantial language-visual inconsistency.

3.2.3 Joint Training Objectives

RynnValue is jointly optimized using three cross-entropy objectives: an absolute temporal-distance loss, a relative temporal-distance loss, and a causal language-modeling loss. Although all three objectives use cross-entropy, the first two operate over distributional temporal bins, whereas the third operates over the language vocabulary.

Temporal targets. Let t_G denote the relabeled completion cutoff and t_i the timestamp of observation I_{t_i} . We define its absolute temporal-distance target as $v_i^* = \max(0, t_G - t_i)$, and the relative target between two consecutively presented observations as $\Delta_i^* = t_{i+1} - t_i$. Accordingly, $\Delta_i^* > 0$ indicates forward temporal progress, whereas $\Delta_i^* < 0$ indicates temporal regression. Here, “consecutive” refers to adjacency in the presented multimodal sequence, not in the original video.

Both continuous targets are transformed with symlog and encoded as two-hot distributions over 256-bin supports, denoted $\mathcal{B}_V$ and $\mathcal{B}_R$ for the absolute and relative supports, respectively.

Absolute temporal-distance loss. The absolute temporal-distance objective is

$$\mathcal{L}_{\text{abs}} = -\frac{\omega}{K} \sum_{i=1}^K \sum_{b=1}^{|\mathcal{B}_V|} [\text{TwoHot}_{\mathcal{B}_V}(v_i^*)]_b \log [\text{softmax}(\mathbf{z}_i^V)]_b, \quad (6)$$

where $\mathbf{z}_i^V$ denotes the logits produced by the absolute-value head, and the decoded continuous prediction is v_i . The sample-level mask ω is set to 1 for instruction-matched examples and 0 for instruction-mismatched examples, for which the original completion cutoff is no longer valid.

Relative temporal-distance loss. The relative temporal-distance objective is

$$\mathcal{L}_{\text{rel}} = -\frac{1}{K-1} \sum_{i=1}^{K-1} \sum_{b=1}^{|\mathcal{B}_R|} [\text{TwoHot}_{\mathcal{B}_R}(\Delta_i^*)]_b \log [\text{softmax}(\mathbf{z}_i^R)]_b, \quad (7)$$

where $\mathbf{z}_i^R$ denotes the logits produced by the relative-value head, and the corresponding decoded prediction is denoted by Δ_i . The absolute objective anchors each observation to task completion, whereas the relative objective captures forward and backward temporal displacement between consecutively presented observations.

Natural-language loss. For natural-language supervision, RynnValue autoregressively predicts the structured output defined in Equation 5 and is optimized using standard token-level cross-entropy loss:

$$\mathcal{L}_{\text{lang}} = -\frac{1}{|\mathcal{Y}|} \sum_{j \in \mathcal{Y}} \log p_{\theta}(y_j | \mathbf{x}, y_{<j}), \quad (8)$$

where $\mathcal{Y}$ denotes the supervised language-token positions. Success prediction is therefore learned through the natural-language objective, without introducing a separate success-classification head.

Joint objective. The final training objective combines the three cross-entropy losses:

$$\mathcal{L} = \mathcal{L}_{\text{abs}} + \mathcal{L}_{\text{rel}} + \lambda_{\text{lang}} \mathcal{L}_{\text{lang}}, \quad \lambda_{\text{lang}} = 2. \quad (9)$$

The absolute and relative losses update their corresponding distributional heads and the visual-language backbone. The LM output projection is kept frozen, while gradients from $\mathcal{L}_{\text{lang}}$ still propagate through the fixed projection to adapt the backbone representations. No stop-gradient operation is applied between either temporal head and the backbone.

3.3 Inference and Reward Interface

Chronological inference. At inference time, the training-time sampling augmentations are disabled and the input observations are arranged in chronological order. The value-isolation attention mask remains active: each absolute or relative query group can attend to its associated visual-language context, while predictions from other query groups remain inaccessible. This prevents previously predicted values from influencing subsequent predictions during inference. RynnValue then decodes the absolute and relative distributional outputs into continuous temporal-distance estimates. The language branch subsequently generates the video description, instruction-video matching judgment, and success judgment, none of which are fed back into the temporal-value predictions.

Temporal-distance potential. RynnValue predicts temporal distance rather than reward directly. Let v_t denote the final temporal-distance estimate for observation I_t , after distributional decoding. The original prediction remains non-negative: a large v_t indicates that more time is required to complete the task, whereas $v_t = 0$ corresponds to the predicted completion boundary. To match the conventional value semantics that *larger is better*, we convert temporal distance into an observation potential by reversing its sign:

$$\Phi_t = \Phi_\theta(I_t, \ell, m) = -v_t, \quad (10)$$

where ℓ and m denote the task instruction and metadata, respectively. Consequently, states before task completion have negative potential, while the potential approaches zero as the agent reaches the goal. This sign reversal preserves the temporal scale rather than normalizing predictions to a task-specific $[0, 1]$ interval.

4 Experiments

We evaluate RynnValue along four axes. First, we benchmark its intrinsic value quality against standardized reward-model evaluation suites (Section 4.2). Second, we analyze how its accuracy scales with data quantity and task diversity (Section 4.3). Third, we qualitatively compare its temporal-value predictions with a strong baseline on held-out trajectories, examining whether the learned value tracks task progress and reacts properly to regressions and failures (Section 4.4). Finally, we deploy RynnValue as a reward interface in real-world reinforcement learning to assess its utility under physical execution noise and embodiment shift (Section 4.5).

4.1 Experimental Setup

Model configuration. In Table 2 we additionally report a RynnValue-4B variant to probe how performance scales with backbone size; all other experiments use the 8B model. Each training sample contains $K = 8$ observations, with eight repeated query tokens for each absolute or relative prediction slot. Both temporal heads are implemented as BroNet [21] residual MLPs with hidden width 4096, depth 8, and ReLU activations. The absolute and relative temporal targets are represented using 256-bin symlog supports over $[0, 512]$ and $[-256, 256]$, respectively. We use value-isolation attention throughout both training and inference.

Optimization. We jointly optimize the absolute temporal-distance, relative temporal-distance, and natural-language objectives as shown in Equation 9. RynnValue is trained with AdamW using a learning rate of 1×10^{-6} , $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1, and $\epsilon = 10^{-8}$. We use a constant learning-rate schedule without warm-up and clip the gradient norm at 100. Training is conducted in bfloat16 with FSDP hybrid sharding and a per-device batch size of 2.

During training, temporal-order shuffling is applied with probability 0.5, and the forward-biased sampling process uses a rewind probability of 0.3. Instruction-mismatch augmentation is applied to 10% of the samples. For mismatched samples, the absolute temporal-distance loss is masked, while the relative and natural-language objectives are retained.

Table 2 Per-dataset trajectory-ranking results on the RBM-EVAL-OOD test suite, measured by Kendall’s τ_a ($\uparrow$). Bold values indicate the best overall results. $\dagger$ denotes the best result among methods trained without explicit trajectory-level preference supervision, i.e., progress/value-only methods. Baseline results are taken from Robometer [16].

Method	USC Franka	USC Koch	USC Trossen	USC xArm	MIT Franka	UTD SO101	Average
GVL [19]	0.250	−0.008	0.292	0.056	0.306	0.300	0.199
VLAC-2B [34]	0.292	0.167	−0.111	0.167	−0.017	−0.033	0.077
VLAC-8B [34]	0.271	0.064	−0.417	0.139	0.072	0.167	0.049
RoboDopamine [26]	0.167	0.175	0.000	0.014	0.220	0.067	0.107
Dopamine-GRM-2.0-8B-Preview [26]	0.479	0.442	0.333	0.431	0.431	0.700	0.453
RoboReward-4B [15]	0.625	0.332	0.333	0.528	0.494	0.700	0.502
RoboReward-8B [15]	0.625	0.264	0.389	0.347	0.396	0.767	0.465
Robometer (RoboReward data) [16]	0.583	0.533	0.646	0.403	0.479	0.667	0.552
ReWiND [33]	−0.125	0.336	0.028	−0.167	0.080	−0.067	0.014
Robometer (RBM-1M) [16]	0.646	0.471	0.653	0.694	0.601	0.867	0.655
Robometer (Progress only) [16]	0.083	0.231	0.333	0.389	0.183	0.533	0.292
RynnValue-4B	0.542	0.488	0.917	0.667 $\dagger$	0.473	0.933$\dagger$	0.670
RynnValue-8B	0.667$\dagger$	0.544$\dagger$	1.000$\dagger$	0.500	0.503 $\dagger$	0.833	0.675$\dagger$

4.2 Benchmark Evaluation

We evaluate RynnValue on the trajectory-ranking track of the RBM-EVAL-OOD test suite introduced by Robometer [16]. The benchmark contains 976 trajectories from six out-of-distribution datasets spanning different institutions, robot embodiments, camera viewpoints, and task families. For each task, the benchmark provides trajectories annotated with different execution-quality levels, including failed, suboptimal, and successful executions. Performance is measured by Kendall’s τ_a between the ground-truth trajectory-quality ordering and the ordering induced by the predicted scores.

Because RynnValue predicts temporal distance rather than normalized progress, we score each trajectory by its temporal-distance potential (Equation 10) at the final queried observation, *i.e.*, the negative predicted remaining distance $-v_{\text{end}}$, so that a trajectory closer to task completion receives a higher score. No additional normalization or cross-dataset calibration is required because Kendall’s τ_a depends only on the relative ordering of these scores. We use the absolute temporal-distance output for this evaluation; the relative temporal prediction and natural-language outputs are not used to compute the benchmark metric.

4.2.1 Main Results

As shown in Table 2, RynnValue-8B achieves the highest average Kendall’s τ_a of 0.675, and even the smaller RynnValue-4B reaches 0.670; both exceed the 0.655 obtained by the full Robometer model trained with both progress and trajectory-preference supervision. In terms of the best overall results, RynnValue-8B ranks first on USC Franka, USC Koch, and USC Trossen, while RynnValue-4B attains the top score on UTD SO101. Among methods trained without explicit trajectory-level preference supervision, RynnValue variants attain the best results on all six datasets, improving the strongest preference-free prior average of 0.502 to 0.670 at 4B and 0.675 at 8B. RynnValue also substantially outperforms the progress-only Robometer ablation, which achieves an average score of 0.292. Notably, performance is already strong at 4B and improves only marginally at 8B, suggesting that the gains stem from the temporal-distance formulation and its accompanying training and architectural designs rather than sheer model scale.

These results support temporal distance as an effective supervision target for scaling robotic value models. Unlike explicitly constructed trajectory preferences, temporal-distance labels can be obtained automatically from heterogeneous trajectories while preserving a consistent cost-to-go interpretation across tasks, execution speeds, and robot embodiments. Together with random temporal sampling, order shuffling, and value-isolation attention, this formulation encourages the model to ground its predictions in task-relevant visual evidence rather than dataset-specific progress scales or temporal shortcuts. Most importantly, RynnValue reaches this quality through a substantially simpler recipe: its targets are read directly from trajectory timestamps, without constructing preference pairs or normalizing every task onto a $[0, 1]$ progress scale. That a preference-free,

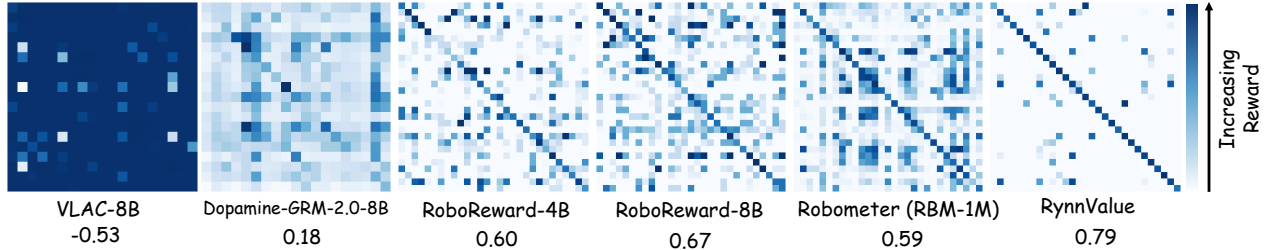


Figure 3 Instruction-trajectory confusion matrices. Each cell shows the predicted reward when an instruction (rows) is paired with a trajectory (columns); a well-grounded model concentrates mass on the diagonal. Values below each matrix report the normalized diagonal margin. All models are re-evaluated under a unified protocol from their publicly released weights.

Table 3 Ablation study on RBM-EVAL-OOD. We report Kendall’s τ_a across six out-of-distribution robot datasets. Shuffle denotes temporal-order shuffling, Isolation denotes value-isolation attention, Language denotes the auxiliary natural-language supervision, Random denotes random temporal sampling, and Relative denotes the relative modeling component.

Variant	Design Components					Kendall’s τ_a						
	Shuffle	Isolation	Language	Random	Relative	USC Franka	USC Koch	USC Trossen	USC xArm	MIT Franka	UTD SO101	Average
w/o Shuffle	✗	✓	✓	✓	✓	0.583	0.090	0.055	0.222	-0.017	0.200	0.189
w/o Isolation	✓	✗	✓	✓	✓	0.583	0.428	0.694	0.389	0.400	0.400	0.482
w/o Language	✓	✓	✗	✓	✓	0.250	0.491	0.819	0.361	0.501	0.800	0.537
Uniform Sampling	✓	✓	✓	✗	✓	0.375	0.400	0.305	0.250	0.310	0.633	0.379
w/o Relative	✓	✓	✓	✓	✗	0.667	0.587	0.639	0.639	0.464	0.767	0.627
Full Model (8B)	✓	✓	✓	✓	✓	0.667	0.544	1.000	0.500	0.503	0.833	0.675

timestamp-derived target can match and even surpass fully preference-supervised training highlights temporal distance as a more scalable route to general-purpose robotic value learning.

4.2.2 Instruction-Trajectory Alignment

To assess whether the predicted reward tracks the specified language goal rather than generic visual progress, we perform an instruction-trajectory alignment analysis: we score every instruction against every trajectory and inspect the resulting matrix, following the evaluation protocol of Robometer [16]. A well-grounded model should assign higher rewards to matched instruction-trajectory pairs along the diagonal and lower rewards to mismatched pairs. As shown in Figure 3, to ensure that all confusion matrices are produced under an identical protocol, we re-evaluate every baseline with its publicly released model weights, and report the resulting diagonal-margin scores. RynnValue produces the clearest diagonal structure and achieves the highest normalized diagonal margin of **0.79**, outperforming the strongest baseline at 0.67. In contrast, several baselines exhibit more diffuse off-diagonal responses. These results show that RynnValue more effectively distinguishes matched from mismatched instruction-trajectory pairs, grounding its temporal-distance estimates in the language-specified goal rather than generic visual progress alone.

4.2.3 Ablation Study

We ablate the principal training and architectural components of RynnValue in Table 3. The full model achieves the highest average Kendall’s τ_a of 0.675. Replacing random temporal sampling with uniform sampling reduces the average score to 0.379. The regular intervals it produces let the model exploit a stereotypical value pattern rather than reason from the observed visual content. Similarly, removing temporal-order shuffling causes the largest degradation, reducing the average score to 0.189, as sequence position again becomes a strong proxy for task progress.

Beyond sampling, removing value-isolation attention decreases performance to 0.482. Taken together, these results validate our complementary shortcut-suppression designs: shuffling prevents progress inference from presentation order, while value isolation prevents prediction queries from extrapolating values across query

groups. We further examine the auxiliary training objectives. Removing natural-language supervision reduces the average score to 0.537, demonstrating that video-description, instruction-matching, and success supervision provide useful semantic grounding. Finally, removing relative temporal-distance supervision yields a score of 0.627. By modeling local forward and backward temporal changes, this objective complements the globally anchored absolute prediction and improves the shared visual representation. This benefit is particularly evident on USC Trossen, where its removal reduces the score from 1.000 to 0.639.

4.3 Scaling Analysis

Having isolated the contribution of each model component, we next ask whether the heterogeneous data recipe itself drives generalization. To disentangle *quantity* from *diversity*, we construct two families of training subsets from the full mixture. In the **episode-volume** family (orange), we fix the full set of training tasks and randomly subsample $\{1\%, 10\%, 25\%, 50\%, 75\%\}$ of episodes within each task. In the **task-diversity** family (blue), we fix per-task episode counts at the full-scale level and randomly subsample the same fractions of tasks. At each fraction, both protocols train on a comparable total number of episodes, ensuring that performance differences reflect diversity rather than data volume. Both families converge to the same full training set at 100%. We train each variant from scratch under identical optimization settings and report the mean absolute temporal-distance error on a held-out validation set whose tasks do not overlap with any training subset.

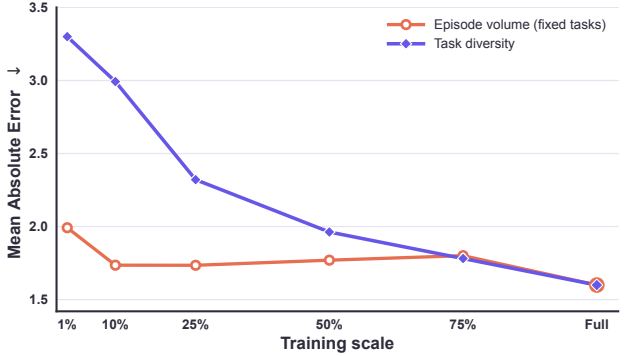


Figure 4 Scaling episode volume vs. task diversity. Mean absolute temporal-distance error on a held-out validation set of unseen tasks. We independently scale two aspects of the training set: (orange) episode count with the full task set fixed, and (blue) task count with per-task episode counts fixed. Both curves converge to the same full-scale training set at 100%. Task diversity yields a consistently steeper error reduction than episode volume, which saturates early.

As shown in Figure 4, episode-volume scaling saturates almost immediately: beyond a small fraction of episodes the error plateaus and does not improve further as more within-task data are added. Task-diversity scaling behaves qualitatively differently: increasing the number of training tasks reduces error monotonically across the entire range, with substantial gains still observable well past the midpoint. This contrast demonstrates that the heterogeneous data recipe contributes not merely additional samples, but the diversity required for general-purpose temporal-value learning: broader task coverage introduces diverse goal structures, visual configurations, and execution patterns that transfer to unseen tasks, whereas repeated observation of the same tasks quickly stops adding signal.

4.4 Temporal-Value Curve Case Study

Figure 5 compares the value curves predicted by RynnValue and Robometer along the same real-world trajectory. Because Robometer outputs normalized progress while RynnValue outputs temporal-distance potential ($\Phi_t = -v_t$), the two scales are not directly comparable; we therefore orient both so that higher values indicate closer proximity to task completion. The visualization reveals three notable differences.

First, RynnValue responds more strongly to trial-and-error behavior and task regression. During the highlighted interval, its potential decreases substantially as the robot moves away from a productive state, whereas Robometer shows a weaker response. This sensitivity is consistent with temporal-order shuffling and value-isolation attention, which encourage predictions to rely on visual evidence rather than sequence position or previously exposed values.

Second, after the robot recovers, RynnValue exhibits a steadier increase toward completion, while Robometer contains longer plateaus and abrupt jumps. This behavior benefits from jointly learning absolute and relative temporal values, which capture global remaining cost and local temporal changes, respectively.

Third, RynnValue remains sensitive near task completion: late disturbances cause an immediate potential

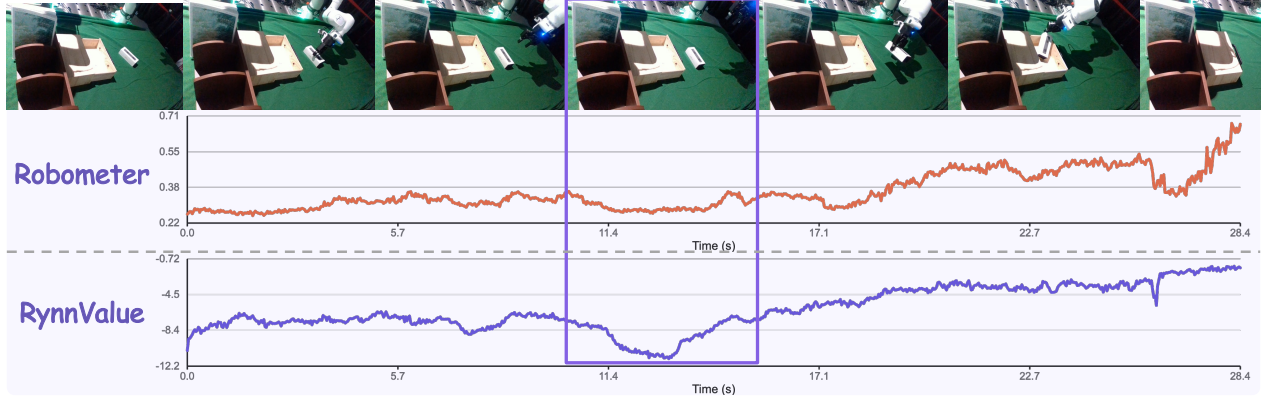


Figure 5 Temporal-value curve comparison on a real-world trajectory. Higher values indicate closer proximity to task completion. The highlighted interval marks a period of task regression where the robot moves away from a productive state; RynnValue responds with a sharp potential drop, whereas Robometer remains relatively flat.



Figure 6 Representative demonstrations for real-world evaluation. Each row shows a sequence of observations from one manipulation task: Each row shows a sequence of observations from one manipulation task. These tasks cover object grasping, spatial manipulation, and articulated-object interaction.

decrease followed by recovery. This avoids premature reward saturation and better distinguishes intermediate progress from actual completion.

4.5 Real-World Policy Learning

Beyond benchmarks, we evaluate whether RynnValue can improve policy learning on a diverse set of challenging real-world manipulation tasks. Since none of these tasks, their manipulated objects, or the workspace scenes appear in the reward-model training corpus, and no target-domain fine-tuning is performed, RynnValue serves as a zero-shot reward annotator. This setting assesses its open-world generalization and its effectiveness for value estimation under physical execution noise.

4.5.1 Experimental Settings

Robot platform. Our experiments are conducted on a dual-arm Franka robot system with wrist-mounted cameras on both arms and two additional third-person cameras capturing the workspace from the left and right sides.

Tasks and evaluation protocol. We evaluate on the following four real-world robotic manipulation tasks as shown in Figure 6:

Table 4 Real-world reinforcement-learning results. We report success rates and the average number of action chunks over successful episodes. Average denotes the unweighted mean success rate across four tasks.

Algorithm	Baseline	Bread Basket Placement		Steak Serving with a Spatula		Box-in-Drawer Placement		Bimanual Box Transfer		Average Success
		Success ↑	Avg. Steps ↓	Success ↑	Avg. Steps ↓	Success ↑	Avg. Steps ↓	Success ↑	Avg. Steps ↓	
Online RL	RynnValue	45.0%	25.9 ± 8.2	75.0%	18.6 ± 13.1	70.0%	27.0 ± 8.1	100.0%	22.8 ± 4.7	72.5%
	Robometer	35.0%	22.7 ± 5.5	45.0%	15.2 ± 2.7	65.0%	27.7 ± 5.8	65.0%	25.6 ± 7.4	52.5%
	Sparse	40.0%	56.0 ± 31.7	45.0%	18.4 ± 4.9	40.0%	27.4 ± 6.6	70.0%	23.5 ± 2.7	48.8%
Offline RL	RynnValue	100.0%	16.8 ± 3.1	90.0%	14.9 ± 4.0	90.0%	14.9 ± 4.0	50.0%	33.6 ± 10.5	82.5%
	Robometer	80.0%	18.9 ± 2.7	80.0%	19.4 ± 7.2	50.0%	27.3 ± 6.3	45.0%	28.7 ± 9.3	63.8%
	Sparse	70.0%	26.1 ± 9.1	20.0%	30.2 ± 3.3	0.0%	–	0.0%	–	22.5%
SFT	–	70.0%	24.8 ± 8.0	25.0%	18.6 ± 6.2	0.0%	–	0.0%	–	23.8%

- **Bread Basket Placement:** The robot places two pieces of bread into a basket.
- **Steak Serving with a Spatula:** The robot uses a spatula to transfer a steak from a pan to a plate.
- **Box-in-Drawer Placement:** The robot places a box into a drawer and then closes the drawer.
- **Bimanual Box Transfer:** Two robot arms collaboratively move a box to a target location.

For each task, we evaluate each policy over 20 trials and report both the success rate and the average number of action chunks over successful episodes. All offline evaluations use fixed initial configurations. During online evaluation, we randomize the initial object configurations for *Bread Basket Placement*, *Steak Serving with a Spatula*, and *Bimanual Box Transfer* to assess robustness under varying initial conditions, while *Box-in-Drawer Placement* uses a fixed reset because the task requires precise gripper control and box-drawer alignment.

Baselines and reward shaping. We compare RynnValue against Robometer, the strongest baseline in our benchmark evaluations, and Sparse Reward, which assigns -1 before task completion and 0 upon successful completion. To ensure a fair comparison, all reward models are used through the same potential-based reward shaping interface:

$$r'_t = \kappa(\gamma\Phi_{t+1} - \Phi_t) + \begin{cases} 0, & \text{if } t = T \text{ and the trajectory is successful,} \\ -1, & \text{otherwise.} \end{cases} \quad (11)$$

where Φ_t denotes the potential predicted by the reward model at step t , γ is the discount factor, and κ controls the strength of the shaping term. We use $\kappa = 0.1$ for RynnValue and $\kappa = 1.0$ for Robometer in both the offline and online experiments. We retain the sparse completion term because reward-model predictions may be noisy, whereas the success label provides a clean and reliable signal for the final task objective. These success labels are manually annotated by human operators during both data collection and online policy training. For Sparse Reward, we set $\Phi_t = 0$ for all t , so the reward is -1 before completion and 0 on the transition that successfully completes the task.

Offline RL with mixed-expertise datasets. First, we evaluate whether RynnValue can effectively recover high-performing policies from mixed-expertise datasets. Specifically, using $\pi_{0.5}$ as the base policy, we apply Implicit Q-Learning (IQL) [14] to train policies on these datasets with reward signals provided by either RynnValue or the baselines.

Online RL with general-purpose reward models. Next, we investigate whether RynnValue can accelerate online policy learning by providing real-time reward specification. We adopt Diffusion Steering via Reinforcement Learning (DSRL) [28] with task-specific policy initialization. Bread Basket Placement and Steak Serving with a Spatula start from their SFT checkpoints, whereas Box-in-Drawer Placement and Bimanual Box Transfer start from the corresponding Robometer-based offline-RL checkpoints. This initialization is held fixed across the online reward variants, whose collected trajectories are specified by either RynnValue or the baselines.

4.5.2 Performance Analysis

As shown in Table 4, RynnValue consistently achieves the highest success rate across all four real-world tasks in both online and offline RL. In online OOD evaluation, it reaches an average success rate of 72.5%, substantially outperforming Robometer at 52.5% and sparse rewards at 48.8%. The improvement is particularly pronounced on *Steak Serving with a Spatula* and *Bimanual Box Transfer*, where RynnValue improves upon Robometer by 30 and 35 percentage points, respectively. Under offline RL, RynnValue achieves an average success rate of 82.5%, compared with 63.8% for Robometer and only 23.8% for the original SFT policy.

RynnValue also provides a strong balance between task success and execution efficiency. On *Bread Basket Placement*, it achieves 100% offline success using 16.8 action chunks on average, compared with 80% and 18.9 chunks for Robometer and 70% and 24.8 chunks for SFT. Similar improvements are observed on *Steak Serving with a Spatula* and *Box-in-Drawer Placement*, where RynnValue reaches 90% success while requiring only 14.9 chunks. Notably, reinforcement learning with RynnValue successfully solves *Box-in-Drawer Placement* and *Bimanual Box Transfer*, for which the SFT policy records no successful executions. These results show that RynnValue enables substantial improvements over the imitation-learning baseline while maintaining efficient task completion.

The online gains are limited for both reward models on *Box-in-Drawer Placement*. Starting from the shared Robometer-trained offline-RL checkpoint with a 50% success rate, online RL reaches 65% with Robometer and 70% with RynnValue. This task requires precise coordination among the gripper, the box, and the drawer geometry. Because the reward models observe only third-person RGB images, visually similar configurations may correspond to substantially different grasp stability and placement alignment, making it difficult to assign well-calibrated intermediate rewards. Consequently, neither reward model yields a substantial online improvement in this visually ambiguous, precision-sensitive setting, although RynnValue remains slightly more effective than Robometer.

5 Conclusion and Future Works

We presented RynnValue, a value foundation model for robotic manipulation that replaces trajectory-internal progress with temporal distance as its supervision target. By treating a state’s directed, goal-conditioned cost-to-go as the learning objective, RynnValue turns heterogeneous data into a single, preference-free value interface: labels are derived directly from timestamps, and disparate embodiments, viewpoints, and task durations are unified without dataset-specific progress normalization. To make temporal-value learning robust at scale, we combined random temporal sampling and temporal-order shuffling with value-isolation attention, suppressing the sampling-interval, sequence-order, and value-extrapolation shortcuts that otherwise leave learned values insensitive to failures and regressions.

Trained without a single preference label, RynnValue surpasses the fully preference-supervised state of the art on RBM-EVAL-OOD and nearly doubles a progress-only counterpart, while generalizing zero-shot across unseen tasks, embodiments, and viewpoints. Converted into dense rewards through potential-based shaping, it improves both online and offline real-world policy learning over strong reward-model and sparse-reward baselines. Together, these results indicate that temporal distance is both a scalable supervision target for value foundation models and a practical reward interface for generalist robot policies.

Looking forward, we aim to extend RynnValue toward broader temporal horizons, richer value semantics, and more diverse embodiments. RynnValue currently estimates temporal distance from a short window of sampled observations, so extending it to longer horizons and streaming inference would broaden its use as an online reward source. Its targets assume an approximately minimum-time objective, and incorporating task-specific costs such as energy, safety, or precision could yield richer value semantics. Finally, we plan to scale RynnValue to a wider range of end-effectors, including dexterous hands, as well as mobile manipulation settings, where long-horizon navigation and interaction must be jointly grounded in the same temporal-value interface.

References

- [1] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibao Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- [2] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y Galliker, et al. $\pi_{0.5}$: A Vision-Language-Action Model with Open-World Generalization. In *Proceedings of the Conference on Robot Learning*, 2025.
- [3] Qingwen Bu, Jisong Cai, Li Chen, Xiuqi Cui, Yan Ding, Siyuan Feng, Xindong He, Xu Huang, et al. AgiBot World Colosseum: A large-scale manipulation platform for scalable and intelligent embodied systems. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2025.
- [4] Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascal, Jade Choghari, Jess Moss, and Thomas Wolf. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. <https://github.com/huggingface/lerobot>, 2024.
- [5] Ronghao Dang, Jiayan Guo, Bohan Hou, Sicong Leng, Kehan Li, Xin Li, Jiangpin Liu, Yunxuan Mao, Zhikai Wang, Yuqian Yuan, et al. RynnBrain: Open embodied foundation models. *arXiv preprint arXiv:2602.14979*, 2026.
- [6] Li Dong, Nan Yang, Wenhui Wang, Furu Wei, Xiaodong Liu, Yu Wang, Jianfeng Gao, Ming Zhou, and Hsiao-Wuen Hon. Unified language model pre-training for natural language understanding and generation. In *Proceedings of the Advances in Neural Information Processing Systems*, 2019.
- [7] Jesse Farebrother, Jordi Orbay, Quan Vuong, Adrien Ali Taïga, Yevgen Chebotar, Ted Xiao, Alex Irpan, Sergey Levine, Pablo Samuel Castro, Aleksandra Faust, et al. Stop regressing: Training value functions via classification for scalable deep RL. In *Proceedings of the International Conference on Machine Learning*, pages 13049–13071, 2024.
- [8] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.
- [9] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, pages 1–7, 2025.
- [10] Ryan Hoque, Peide Huang, David J Yoon, Mouli Sivapurapu, and Jian Zhang. EgoDex: Learning dexterous manipulation from large-scale egocentric video. *arXiv preprint arXiv:2505.11709*, 2025.
- [11] Bohan Hou, Haoqiang Lin, Xuemeng Song, Haokun Wen, Meng Liu, Yupeng Hu, and Xiangyu Zhao. FiRE: Enhancing MLLMs with fine-grained context learning for complex image retrieval. In *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 803–812, 2025.
- [12] Physical Intelligence, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Jared DiCarlo, et al. $\pi_{0.6}^*$: A VLA that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.
- [13] Tao Jiang, Tianyuan Yuan, Yicheng Liu, Chenhao Lu, Jianning Cui, Xiao Liu, Shuiqi Cheng, Jiyang Gao, Huazhe Xu, and Hang Zhao. Galaxea open-world dataset and G0 dual-system VLA model. *arXiv preprint arXiv:2509.00576*, 2025.
- [14] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit Q-learning. In *Proceedings of the International Conference on Learning Representations*, 2022.
- [15] Tony Lee, Andrew Wagenmaker, Karl Pertsch, Percy Liang, Sergey Levine, and Chelsea Finn. RoboReward: General-purpose vision-language reward models for robotics. *arXiv preprint arXiv:2601.00675*, 2026.

- [16] Anthony Liang, Yigit Korkmaz, Jiahui Zhang, Minyoung Hwang, Abrar Anwar, Sidhant Kaushik, Aditya Shah, Alex S Huang, Luke Zettlemoyer, Dieter Fox, et al. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons. *arXiv preprint arXiv:2603.02115*, 2026.
- [17] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. RDT-1B: a diffusion foundation model for bimanual manipulation. In *Proceedings of the International Conference on Learning Representations*, pages 29982–30009, 2025.
- [18] Yuyang Liu, Chuan Wen, Yihang Hu, Dinesh Jayaraman, and Yang Gao. TimeRewarder: Learning dense reward from passive videos via frame-wise temporal distance. *arXiv preprint arXiv:2509.26627*, 2025.
- [19] Yecheng Jason Ma, Joey Hejna, Ayzaan Wahid, Chuyuan Fu, Dhruv Shah, Jacky Liang, Zhuo Xu, Sean Kirmani, Peng Xu, Danny Driess, Ted Xiao, Jonathan Tompson, Osbert Bastani, Dinesh Jayaraman, Wenhao Yu, Tingnan Zhang, Dorsa Sadigh, and Fei Xia. Vision language models are in-context value learners. In *Proceedings of the International Conference on Learning Representations*, 2025.
- [20] Yao Mu, Tianxing Chen, Zanzin Chen, Shijia Peng, Zhiqian Lan, Zeyu Gao, Zhixuan Liang, Qiaojun Yu, Yude Zou, Mingkun Xu, et al. RoboTwin: Dual-arm robot benchmark with generative digital twins. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 27649–27660, 2025.
- [21] Michal Nauman, Mateusz Ostaszewski, Krzysztof Jankowski, Piotr Mił oś, and Marek Cygan. Bigger, Regularized, Optimistic: scaling for compute and sample efficient continuous control. In *Proceedings of the Advances in Neural Information Processing Systems*, pages 113038–113071, 2024.
- [22] Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the International Conference on Machine Learning*, pages 278–287, 1999.
- [23] Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, and et.al. Open X-Embodiment: Robotic learning datasets and RT-X models. In *Proceedings of the IEEE International Conference on Robotics and Automation*, pages 6892–6903, 2023.
- [24] Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.
- [25] Tom Schaul, Daniel Horgan, Karol Gregor, and David Silver. Universal value function approximators. In *Proceedings of the International Conference on Machine Learning*, pages 1312–1320, 2015.
- [26] Huajie Tan, Sixiang Chen, Yijie Xu, Zixiao Wang, Yuheng Ji, Cheng Chi, Yaoxu Lyu, Zhongxia Zhao, Xian-sheng Chen, Peterson Co, et al. Robo-Dopamine: General process reward modeling for high-precision robotic manipulation. *arXiv preprint arXiv:2512.23703*, 2025.
- [27] Yang Tian, Yuyin Yang, Yiman Xie, Zetao Cai, Xu Shi, Ning Gao, Hangxu Liu, Xuekun Jiang, Zherui Qiu, Feng Yuan, et al. InternData-A1: Pioneering high-fidelity synthetic data for pre-training generalist policy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 976–985, 2026.
- [28] Andrew Wagenmaker, Mitsuhiko Nakamoto, Yunchu Zhang, Seohong Park, Waleed Yagoub, Anusha Nagabandi, Abhishek Gupta, and Sergey Levine. Steering your diffusion policy with latent space reinforcement learning. In *Proceedings of the Conference on Robot Learning*, 2025.
- [29] Kun Wu, Chengkai Hou, Jiaming Liu, Zhengping Che, Xiaozhu Ju, Zhuqin Yang, Meng Li, Yinuo Zhao, Zhiyuan Xu, Guang Yang, et al. RoboMIND: Benchmark on multi-embodiment intelligence normative data for robot manipulation. *arXiv preprint arXiv:2412.13877*, 2024.
- [30] Shihan Wu, Xuecheng Liu, Shaoxuan Xie, Pengwei Wang, Xinghang Li, Bowen Yang, Zhe Li, Kai Zhu, Hongyu Wu, Yiheng Liu, et al. RoboCOIN: An open-sourced bimanual robotic data collection for integrated manipulation. *arXiv preprint arXiv:2511.17441*, 2025.
- [31] Pengzhi Yang, Xinyu Wang, Pengyu Jing, Kehan Wen, Yiduo Qu, Zhenhao Huang, Minghao Fu, Xin Liu, Yaheng Shen, and Fan Shi. RARM: Confidence-gated progress reward modeling for RL in manipulation. *arXiv preprint arXiv:2606.22027*, 2026.
- [32] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. XLNet: Generalized autoregressive pretraining for language understanding. In *Proceedings of the Advances in Neural Information Processing Systems*, 2019.

- [33] Jiahui Zhang, Yusen Luo, Abrar Anwar, Sumedh Anand Sontakke, Joseph J Lim, Jesse Thomason, Erdem Biyik, and Jesse Zhang. ReWiND: Language-guided rewards teach robot policies without new demonstrations. In *Proceedings of the Conference on Robot Learning*, 2025.
- [34] Qi Zhang, Shaopeng Zhai, Shengzhe Zhang, Litao Liu, Tianyi Zhang, Fuxian Huang, and Ming Zhou. A generalist pair-wise progress critic model for vision-language-action robots. In *Proceedings of the International Conference on Machine Learning*, 2026.
- [35] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, et al. X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*, 2025.

Appendix

A Data Curation for Heterogeneous Robot Corpora

Learning temporal distance requires more than a large collection of robot videos: each trajectory segment must be associated with a meaningful language goal and must describe task-relevant evolution toward that goal. Raw robot corpora do not always satisfy these conditions. Their annotations may contain placeholders, data-quality tags, truncated commands, or descriptions of pure robot motion. We therefore apply a source-aware curation pipeline to Open X-Embodiment (OXE), InternData-A1 (InternA1), Galaxea Open-World (Galaxea), and RoboCOIN before constructing temporal-distance targets. Since OXE and InternA1 are counted in episodes, whereas Galaxea and RoboCOIN are counted in annotated subtask segments, we refer to their entries collectively as *trajectory units*.

Instruction validation. We first retain only instructions that specify a well-formed and actionable robot goal. Source-specific usable-instruction sets are used when canonical task vocabularies are available, while malformed strings, placeholders, data-quality metadata, and ambiguous noun-only labels are removed. We also discard CJK annotations from sources whose task interface is defined in English. This stage removes annotation artifacts without altering the semantics of valid manipulation tasks.

Action-relevance filtering. Some mobile-manipulation trajectories contain segments that describe only locomotion or approach behavior, without a manipulation objective. This issue is most prominent in Galaxea. We identify such segments when motion predicates such as moving, approaching, turning, or walking occur without any manipulation predicate, and remove them from temporal-distance training. This ensures that a retained segment describes observable progress toward an interaction goal rather than incidental platform motion.

Table 5 gives representative examples of the annotation issues addressed by the pipeline, spanning several qualitatively different failure modes: malformed language, dataset metadata, task-irrelevant motion.

Table 5 Representative annotation issues addressed by data curation. Non-English examples are described rather than reproduced verbatim to avoid introducing non-Roman fonts into the manuscript.

Issue	Representative annotation	Source
Non-English task annotation	Chinese-language manipulation instruction	Galaxea
Placeholder or truncated label	P, shirts, undefined	InternA1
Data-quality metadata	no robot motion, skip frame	OXE
Pure-motion instruction	move to the table, approach the cabinet	Galaxea

Table 6 summarizes the quantitative effect of the complete pipeline. Overall, curation retains 1,436,150 of 1,722,966 trajectory units (83.35%) while preserving 192,989 of 194,967 unique instructions (98.99%). This gap between unit retention and instruction retention indicates that the removed volume is concentrated in invalid annotations rather than in rare valid tasks.

The resulting mixture retains broad task coverage while reducing several sources of supervision bias. Under a diagnostic first-verb grouping, the miscellaneous *other* category decreases from 4.54% to 1.37%, reflecting the removal of malformed and non-actionable annotations. The dominant *pick-place* category decreases from 43.38% to 33.61% after filtering invalid OXE annotations. The three largest skill groups still account for 68.21% of the curated data, showing that the pipeline does not impose an artificial uniform distribution; instead, it improves task relevance and limits template domination while preserving the natural structure of the robot data.

As an additional post-hoc diagnostic, we normalize the first verb of each instruction into an atomic skill, merging morphological variants such as *pickup*, *picking*, and *picked*. The number of atomic verb categories decreases from 640 in the raw annotations to 549 after curation. The removed categories correspond mainly to empty tokens, termination markers, malformed strings, and other annotation artifacts. This atomic-skill

Table 6 Summary of source-specific data curation. OXE and InternA1 are counted in episodes, while Galaxea and RoboCOIN are counted in annotated subtask segments. The total therefore denotes an aggregate number of trajectory units rather than a homogeneous episode count.

Source	Raw units	Curated units	Unit retention	Raw instructions	Curated instructions	Instruction retention
OXE (episode)	961,253	693,037	72.10%	180,290	180,090	99.89%
InternA1 (episode)	320,910	320,905	99.99%	350	348	99.43%
Galaxea (segment)	97,287	78,692	80.89%	12,685	10,909	86.00%
RoboCOIN (segment)	343,516	343,516	100.00%	1,642	1,642	100.00%
Total	1,722,966	1,436,150	83.35%	194,967	192,989	98.99%

analysis is used only to diagnose the effect of data curation; it does not define the temporal-distance targets or alter the training sampler.

B Real-World Experiment Details

This section provides the implementation details needed to reproduce the robotic policy-learning experiments in Section 4.5. We first define the interfaces among the VLA policy and the auxiliary RL networks, and then describe the tasks and datasets, robotic platform, evaluation protocol, initialization, offline and online RL procedures, and compute resources.

B.1 Implementation Inputs and Outputs

The policy-learning pipeline contains three model interfaces. Throughout this section, o_t denotes the multi-camera visual observation at policy-decision step t , and a_t denotes the corresponding executable action chunk. No component receives a proprioceptive or robot-state vector.

VLA policy. The VLA consumes camera images resized to 224×224 , image-validity masks, the task instruction, and the flow-matching timestep. It outputs an action chunk with horizon $H = 16$ and padded action dimension $d_a = 32$.

IQL critic and value. The offline RL networks consume the current and next multi-camera observations together with the demonstrated action chunk. They estimate $Q(o, a)$ and $V(o)$, from which we compute the advantage weight used to refine the VLA.

DSRL actor-critic. The online RL networks consume only multi-camera observations at 64×64 . The actor predicts a latent-noise chunk $z \in [-1, 1]^{16 \times 32}$, the critics estimate its Q-values, and automatic entropy tuning optimizes the temperature α .

B.2 Tasks and Datasets

We evaluate all methods on 4 robotic manipulation tasks. For each task, we collect approximately 100 successful trajectories and retain every unsuccessful attempt encountered during the same collection process, yielding a mixed-expertise dataset. Table 7 reports the task instructions and dataset statistics.

Table 7 Real-world task instructions and dataset statistics. Success rate is computed over all collected trajectories for each task.

Task	Language instruction	Trajectories			Success rate
		Success	Failure	Total	
Bread Basket Placement	“Put the two pieces of bread in the basket.”	99	4	103	96.1%
Steak Serving with a Spatula	“Move the steak from the pan to the plate.”	98	4	102	96.1%
Box-in-Drawer Placement	“Put the box in the drawer and close it.”	101	3	104	97.1%
Bimanual Box Transfer	“Move the box from the right side to the left side.”	100	1	101	99.0%
Total	—	398	12	410	97.1%

B.3 Robotic Platform and Evaluation Protocol

Camera configuration. The dual-arm Franka platform is instrumented with 4 Intel RealSense cameras: 2 D435 third-person cameras, denoted `left_side` and `right_side`, and 2 D405 wrist-mounted cameras, denoted `left_wrist` and `right_wrist`. Images are resized with aspect-preserving padding to 224×224 for the VLA and IQL critic; the DSRL visual encoder receives inputs downsampled to 64×64 . Table 8 lists the exact streams used by each pathway.

Table 8 Camera streams used by the policy and RL encoders. Entries list the exact streams provided to each pathway.

Pathway	Single-arm tasks	Bimanual Box Transfer
VLA	<code>left_side</code> , <code>left_wrist</code>	<code>left_side</code> , <code>left_wrist</code> , <code>right_wrist</code>
IQL critic	<code>left_side</code> , <code>left_wrist</code>	<code>left_side</code> , <code>left_wrist</code> , <code>right_wrist</code>
DSRL actor-critic	<code>left_side</code> , <code>left_wrist</code>	<code>left_side</code> , <code>right_side</code> , <code>left_wrist</code> , <code>right_wrist</code>

Control and action representation. The low-level controller executes absolute joint-position commands and relative gripper commands at 10 Hz. A single-arm action contains seven absolute joint-position values and one relative gripper value; a bimanual action contains 14 absolute joint-position values and 2 relative gripper values. These task-specific actions are padded to $d_a = 32$. Each VLA prediction contains an action chunk with horizon $H = 16$. During online DSRL, the latent action has shape $H \times d_z$ with $d_z = 32$, and the VLA is queried after every $q = 10$ executed low-level control steps.

Reset, success, and termination. Environment resets are performed by an operator, who repositions the robot and objects and confirms that the next episode may begin. All offline-policy evaluations use fixed initial configurations. During online evaluation, Bread Basket Placement, Steak Serving with a Spatula, and Bimanual Box Transfer use randomized initial-object configurations, whereas Box-in-Drawer Placement uses a fixed reset because the task requires precise gripper control and box-drawer alignment. The operator records a binary task-success label. An episode terminates when success or failure is confirmed or when it reaches the maximum horizon of 600 low-level control steps. Reward models provide only the shaping potential; they do not determine success or termination.

Reward-model inference. RynnValue and Robometer are deployed through the same scoring interface. For each collected trajectory, the reward service receives the task instruction and the RGB sequence from the `right_side` third-person camera. Both models operate in a causal, history-conditioned scoring mode. Given a trajectory containing T RGB observations, for each step $t = 1, \dots, T$, the service scores the observation history $o_{1:t}$, uniformly subsampled to 4 frames (rather than the 8 used during training, to match Robometer’s 4-frame inference protocol) before inference. Robometer decodes its native normalized progress $p_t \in [0, 1]$ as the expectation over ten progress bins and uses $\Phi_t^{\text{Robo}} = p_t$. During reward relabeling and online RL, RynnValue uses only its absolute temporal-distance head and does not invoke the language-generation branch. Following

the notation in the main text, we convert its predicted remaining time v_t into a higher-is-better potential using $\Phi_t = -v_t$. Each complete trajectory is scored once after collection, and the resulting potential sequence is sampled at policy-decision boundaries to construct the shaping rewards. Reward relabeling therefore operates at the action-chunk granularity used by the RL algorithm, rather than at every low-level control step.

B.4 Policy and RL Initialization

Supervised fine-tuning (SFT), offline IQL, and online DSRL all use the same flow-matching vision-language-action policy, $\pi_{0.5}$. Sharing this backbone isolates the effect of the learning algorithm and reward specification.

RL networks. The auxiliary RL networks are small relative to the VLA. Offline IQL uses a ResNet-18 visual encoder with GroupNorm, spatial softmax, and a 50-dimensional bottleneck, followed by MLP heads with hidden dimensions (256, 256). It maintains 2 Q-functions and a separate value function. Online DSRL uses a 4-layer convolutional visual encoder with 32 channels per layer, GroupNorm, spatial softmax, and a 50-dimensional bottleneck. Its actor and critic MLPs have hidden dimensions (128, 128, 128); the critic ensemble contains ten Q-functions, and the actor is a tanh-squashed Gaussian distribution over latent noise.

Initialization and training stages. SFT and IQL are independent, single-stage fine-tuning procedures initialized from the same pretrained $\pi_{0.5}$ checkpoint; IQL is not initialized from SFT. SFT optimizes the standard flow-matching behavior-cloning objective, whereas IQL uses advantage-weighted flow matching. Online DSRL uses task-specific initialization: Bread Basket Placement and Steak Serving with a Spatula start from their SFT checkpoints, while Box-in-Drawer Placement and Bimanual Box Transfer start from the corresponding Robometer-based offline-RL checkpoints. The initial checkpoint for each task is held fixed across all online reward variants. During DSRL, the initialized VLA remains frozen and serves only as a noise-conditioned action decoder; the latent policy, critic ensemble, and entropy temperature are optimized.

B.5 Offline RL with IQL

Mixed-expertise dataset construction. For each task j , we construct a mixed-expertise offline dataset by combining all successful and unsuccessful trajectories:

$$\mathcal{D}_j = \mathcal{D}_{j,\text{succ}} \cup \mathcal{D}_{j,\text{fail}}. \quad (12)$$

Each trajectory is segmented at policy-decision boundaries into transitions (o_h, a_h, o_{h+1}, m_h) , where a_h is an action chunk and $m_h \in \{0, 1\}$ is the bootstrap mask. We set $m_h = 0$ for terminal transitions and $m_h = 1$ otherwise.

All reward variants share the same transitions, task-outcome labels, policy initialization, and optimization configuration. We relabel this common dataset with the potential produced by RynnValue or the corresponding baseline reward model; only the potential source and shaping coefficient differ across variants.

Offline reward construction. For a transition observation o_h , we write $\Phi_h = -v_h$ for its shaping potential, where v_h is RynnValue’s decoded absolute temporal-distance prediction for o_h (Equation 10); the instruction ℓ and metadata m are omitted for brevity. The sparse task reward is

$$r_h^{\text{sparse}} = \begin{cases} 0, & \text{if the task is completed after executing } a_h, \\ -1, & \text{otherwise.} \end{cases} \quad (13)$$

Thus, every transition before completion receives -1 , while the transition that completes the task receives 0 . All transitions in an unsuccessful trajectory receive -1 .

The offline potential-based shaping reward is

$$r_h^{\text{shape}} = \gamma_{\text{off}} \Phi_{h+1} - \Phi_h, \quad (14)$$

where γ_{off} is the offline RL discount factor. The final reward used to train IQL is

$$r_h^{\text{off}} = r_h^{\text{sparse}} + \kappa r_h^{\text{shape}}, \quad (15)$$

Table 9 Offline IQL and SFT hyperparameters. All IQL reward variants share the same mixed-expertise dataset, sparse task reward, policy initialization, and optimization configuration. SFT uses the same policy optimizer and learning-rate schedule and is also trained for 10,000 steps per task.

Hyperparameter	Value
Base policy	$\pi_{0.5}$ with flow matching; action dimension 32
Action horizon H	16
Batch size	64
Policy optimizer	AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, weight decay 10^{-10} , and gradient-norm clipping at 1.0
Learning-rate schedule	Cosine decay with 2,000 linear warm-up steps
Peak / final policy learning rate	$3 \times 10^{-5} / 3 \times 10^{-6}$
Policy EMA decay	0.99
Training steps per task	10,000
Critic / value optimizer	Adam with a learning rate of 3×10^{-4}
Offline discount γ_{off}	0.99
Target update rate ρ_{off}	0.005
Expectile parameter τ_e	0.8
Advantage temperature β	10.0
Maximum advantage weight w_{max}	100
Number of Q-functions K_{IQL}	2; minimum aggregation
Critic and value encoder	ResNet-18 with GroupNorm and spatial softmax; 50-dimensional bottleneck
Critic and value hidden dimensions	(256, 256)
Number of critic cameras	2 for single-arm tasks and 3 for the bimanual task
Critic input resolution	224×224 using the VLA preprocessing pipeline
Policy warm-up N_{warm}	200 optimization steps with $w(o, a) = 1$
Sparse task reward	-1 before task completion and 0 upon task completion
Potential-based shaping reward	$r_h^{\text{shape}} = \gamma_{\text{off}} \Phi_{h+1} - \Phi_h$
Shaping coefficient κ	0.1 for RynnValue and 1.0 for Robometer; fixed across tasks
Sparse-reward baseline	$\kappa = 0$
Image augmentation	Random cropping applied to both current and next observations; no color jitter

where $\kappa \geq 0$ is the shaping coefficient used in the main text. We set $\kappa = 0.1$ for RynnValue, $\kappa = 1.0$ for Robometer, and $\kappa = 0$ for the sparse-reward baseline. These values are fixed across tasks.

IQL optimization and VLA refinement. We use the standard IQL objective [14]: the value function is trained by expectile regression, each Q-function uses a one-step temporal-difference target, and the 2 target Q-functions are aggregated by their minimum. Target Q-networks are updated by Polyak averaging. Rather than introducing a separate IQL policy, we retain the flow-matching VLA and weight its training loss by

$$w(o, a) = \min \left\{ \exp \left(\beta \left[\min_k Q_{\theta_k}(o, a) - V_\psi(o) \right] \right), w_{\text{max}} \right\}. \quad (16)$$

For the first N_{warm} optimization steps, we set $w(o, a) = 1$; thereafter, demonstrated actions with larger estimated advantages receive greater weight. Table 9 reports the complete architecture and optimization configuration.

B.6 Online RL with DSRL

Latent policy optimization. For online RL, we use Diffusion Steering via Reinforcement Learning (DSRL) [28]. The task-specific VLA checkpoint described above is frozen and used as a noise-conditioned action decoder:

$$a_t = \pi_{0.5}(o_t, z_t). \quad (17)$$

A Soft Actor-Critic (SAC) policy $\mu_\varphi(z_t | o_t)$ is trained over the latent noise z_t , rather than directly over executable robot actions. The latent action space is

$$z_t \in [-1, 1]^{H \times d_z}, \quad (18)$$

where $H = 16$ is the action horizon and $d_z = 32$ is the per-step latent dimension.

SAC optimization. We use the standard entropy-regularized SAC objective in the latent action space. Critic targets use the mean of ten target Q-functions, the actor trades off the ensemble-mean Q-value against policy entropy, and the temperature is optimized automatically toward target entropy $\overline{\mathcal{H}} = -\dim(z)$. Target critics are updated by Polyak averaging. Table 10 reports the complete architecture and optimization configuration.

Online reward construction. Online DSRL uses the same sparse task-reward convention as Equation 13: each transition receives -1 until task completion, and the completing transition receives 0 .

Let γ_s denote the per-environment-step shaping discount. Because each policy decision executes q low-level control steps, the corresponding chunk-level discount is

$$\gamma_c = \gamma_s^q, \quad (19)$$

and the online shaping reward is

$$r_t^{\text{shape}} = \begin{cases} \gamma_c \Phi_{t+1} - \Phi_t, & \text{if } o_{t+1} \text{ is nonterminal,} \\ -\Phi_t, & \text{if } o_{t+1} \text{ is terminal.} \end{cases} \quad (20)$$

We use the shaping coefficient κ defined in the main text, setting $\kappa = 0.1$ for RynnValue, $\kappa = 1.0$ for Robometer, and $\kappa = 0$ for the sparse-reward baseline. These values are fixed across tasks, and all variants otherwise share the same reward definition and online optimization configuration. The final reward used by SAC is

$$r_t^{\text{on}} = r_t^{\text{sparse}} + \kappa r_t^{\text{shape}}. \quad (21)$$

Table 10 Online DSRL hyperparameters. SAC operates in the latent space of the frozen VLA, which decodes latent variables into executable action chunks. All reward variants share the same optimization configuration and differ only in the potential source and shaping coefficient.

Hyperparameter	Value
Base policy	Frozen SFT checkpoint for Bread Basket Placement and Steak Serving with a Spatula; frozen Robometer offline-RL checkpoint for Box-in-Drawer Placement and Bimanual Box Transfer
Latent action space	$z \in [-1, 1]^{H \times d_z}$, with $H = 16$ and $d_z = 32$
RL algorithm	SAC with automatic entropy tuning and initial temperature $\alpha_0 = 1.0$
Actor optimizer	Adam with a learning rate of 1×10^{-4}
Critic optimizer	Adam with a learning rate of 3×10^{-4}
Temperature optimizer	Adam with a learning rate of 3×10^{-4}
Gradient clipping	None
Target entropy $\mathcal{H}$	$-\dim(z)$
Online SAC discount γ_{on}	0.999
Target update rate ρ_{on}	0.005
Number of Q-functions K_{SAC}	10; mean aggregation
Actor and critic hidden dimensions	(128, 128, 128)
Image encoder	Four-layer CNN with 32 channels per layer, strides (2, 1, 1, 1), VALID padding, GroupNorm, spatial softmax, and a 50-dimensional bottleneck
SAC input resolution	64×64
Batch size	256
Update-to-data ratio	100
Training length	6,000 training steps
Online rollout trajectories	60 per task
Replay-buffer capacity	$\max(\text{training steps}/\text{UTD}, 10^4) = 10^4$
Update frequency	After each episode
Exploration warm-up	$N_{\text{noise}} = 2$ episodes with Gaussian noise standard deviation $\sigma = 0.1$
Minimum replay size N_{start}	200 transitions
Maximum episode length	600 environment steps
Policy-decision frequency	Low-level control at 10 Hz, with one policy decision every $q = 10$ environment steps
Number of DSRL cameras	2 for single-arm tasks and 4 for the bimanual task
Sparse task reward	-1 before task completion and 0 upon task completion
Shaping coefficient κ	0.1 for RynnValue, 1.0 for Robometer, and 0 for the sparse-reward baseline; fixed across tasks
Per-step shaping discount γ_s	0.999

B.7 Compute Resources

Offline IQL and SFT each use 2 GPUs with 80GB of memory each. IQL assigns one FSDP device to each replica, whereas SFT uses two-way model sharding. Training one task requires approximately 16 hours for offline IQL and 6 hours for SFT. Online DSRL runs on a single x86 server equipped with two Intel Xeon Platinum 8575C processors (48 physical cores per socket with hyper-threading enabled; 192 logical cores in total), 1.5 TiB of system memory, and eight NVIDIA GeForce RTX 5090 GPUs with 32 GB of memory each. The server has a dual-socket NUMA topology and runs NVIDIA driver 570.211.01 with CUDA 12.8. Each task uses 6,000 training steps and 60 online rollout trajectories, requiring approximately 1.5 hours. All online DSRL experiments use single-node execution.